

## PEB 结构深度解析与内存定位：API 之外定位 kernel32.dll

### ● 为什么 GetModuleHandle 不可信（红队视角）

之前写注入器时调用过 `GetModuleHandleA("kernel32.dll")` 来获取基址，但有条铁律：杀软/EDR 的 Hook，本质上就是在 `GetModuleHandle` 的代码入口处插了一根“监控线”。

只要程序调用了 `GetModuleHandle`，EDR 会立刻记录：

- 我们访问了哪个 DLL？
- 我们的调用栈是怎样的？
- 我们紧接着用它做了什么？

**解决办法：**绕过这个函数，直接去内存里把它要返回的那个值手工抄出来。

而这个“值”（`kernel32.dll` 的基址），就记录在 Windows 内核为每个进程维护的一张“户口本”里——这个户口本的名字，叫 PEB（进程环境块）。

### ● 找到 PEB 进程环境块的门牌号（`gs:[0x60]`）

在 64 位 Windows 下，当前线程有个专用的寄存器叫 `gs`。它指向当前线程的 TEB（线程环境块）。

#### 为什么是 `gs`？

它相当于每个线程有一个“自己的专用指针”，永远指向它自己的结构体。

在 TEB 的偏移 `0x60` 处，存放着一个指针，它指向本进程的 PEB。

> 物理意义：不需要调用任何 API，CPU 直接提供了 PEB 进程环境块的地址。

在 C++ 中的写法是：`uintptr_t pPeb = __readgsqword(0x60);`

### ● 找到“已加载模块清单”(InMemoryOrderModuleList)

我们现在手里有了 PEB 的地址。

但我们关心的不是 PEB 本身，而是它里面记录“哪些 DLL 被加载了”的清单。

偏移路径（必须记住但不需要背，只需要理解“跳转流程”）：

- ✧ `PEB+0x10` → 指向 `PEB_LDR_DATA` 结构体（可以理解为“模块清单管理员”）。
  - ✧ `PEB_LDR_DATA+0x20` → 指向 `InMemoryOrderModuleList`（这是双向链表的“表头”）。
- 至此，我们拿到了链表的“起点”。但是链表的每个节点，并不是直接放模块信息，而是一个“挂号单”，叫做 `LIST_ENTRY`。

### ● 从“挂号单”拿到完整档案(ContainingRecord)

链表里存的是 `LIST_ENTRY`（它只包含前向指针 `Flink` 和后向指针 `Blink`）。

而真正存放模块基址、模块名的结构体，是 `LDR_DATA_TABLE_ENTRY`。

`LIST_ENTRY` 是嵌入在这个结构体里面的。

想要从挂号单找档案，就得知道“挂号单在档案里偏移了多少字节”。

这就是 `CONTAINING_RECORD` 宏帮我们计算的事情。

> 物理意义：我们知道张纸条被夹在第 30 页的书里。现在有人只把纸条递给我们，我们通过“减去 30 页的厚度”反推出书的位置。

**关键偏移：**

| 字段                          | 在 <code>LDR_DATA_TABLE_ENTRY</code> 中的偏移 |
|-----------------------------|------------------------------------------|
| <code>DllBase</code> （模块基址） | <code>0x30</code>                        |

|                                 |      |
|---------------------------------|------|
| BaseDllName.Buffer (名字的指针)      | 0x48 |
| BaseDllName.Length (名字长度, 单位字节) | 0x50 |

### ● 完整的遍历流程图（按顺序执行即可）

只需要按这个顺序写代码，就能完全不依赖 GetModuleHandle 打印出 kernel32.dll 的基址：

- 读取 gs:[0x60] → 拿到 pPeb。
- 从 pPeb + 0x10 读 → 拿到 pLdr。
- 从 pLdr + 0x20 读 → 得到链表头节点 pListHead。
- 让 pEntry 等于链表头节点的前向指针 (Flink)，即\*(uintptr\_t\*)pListHead。
- 进入循环：如果 pEntry != pListHead，继续。
- 用 CONTAINING\_RECORD 反推出 LDR\_DATA\_TABLE\_ENTRY 的地址。
- 从该地址偏移 0x30 处读取基址，偏移 0x48 处读取名字。
- 如果名字是"kernel32.dll"，打印基址并退出循环。
- 跳到当前节点的前向指针，继续遍历。

### ● 本章小结

目标：不调用 GetModuleHandle，用 gs:[0x60]手工定位 kernel32.dll。

收获：理解了 CONTAINING\_RECORD 的本质是一个“指针偏移反推法”。

红队意义：当 EDR 在 GetModuleHandle 上设钩子时，我们这段代码仍能正常工作（因为它完全不触发用户态 API 调用）。